

Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias

Ingeniería en Ciencias de Datos y Matemáticas

Reporte Técnico Detallado: Etapa 1 del Reto - Gestor de Identidades (Desarrollo en PHP)

Socio Formador: Casa Monarca

Profesores: Raúl Gómez, Anas Wajid y Alberto F. Martínez

Equipo de Trabajo: Grupo 603

Integrantes:

Alberto Rodríguez Reyes (A01383805)

Diego Alberto Rodríguez Ruiz (A01571638)

Rubén Jiménez Sarmiento (A01286256)

Hector Alonso Flores Melendez (A01571545)

Valeria García Hernández (A01742811)

Lugar y Fecha: Monterrey, Nuevo León; 11 de junio de 2026

Índice de contenidos

1. **Resumen Ejecutivo (Abstract)**
2. **Introducción y Planteamiento del Problema**
3. **Marco de Referencia**
 - 3.1. Criptografía de Clave Pública y Estándares Internacionales

- 3.2. Soluciones Tecnológicas en la Literatura: Pycryptodome
- 3.3. Casos de Estudio Reales: El Sistema de Identidades del SAT
- 4. **Especificaciones Técnicas del Desarrollo**
 - 4.1. Arquitectura del Backend (PHP)
 - 4.2. Seguridad y Criptografía
- 5. **Diseño y Ejecución de Pruebas Funcionales**
 - 5.1. Metodología de Pruebas
 - 5.2. Justificación de Decisiones Tecnológicas y de Infraestructura
 - 5.3. Entorno de Pruebas
- 6. **Casos de Prueba**
 - (Casos del 1 al 15)
 - 6.1. Tabla de Casos de Prueba Expandida
- 7. **Análisis de Resultados y Conclusiones**
- 8. **Referencias Bibliográficas**
- 9. **Anexos**
 - Anexo A: Diccionario de Datos de las Tablas Críticas de Seguridad
 - Anexo B: Fragmento de Código Fuente Crítico - Proceso de Handshake y Verificación Criptográfica de Sesión

Índice de tablas

Tabla 1. Fortaleza de clave en RSA comparado con algoritmos simétricos	2
Tabla 2. Matriz de validación y casos de uso del Gestor de Identidades	7

Índice de figuras

Fig. 1. Diagrama de la Arquitectura de Seguridad en Capas (elaboración propia)	4
Fig. 2. Interfaz de inicio de sesión y validación de roles (elaboración propia)	6

(falta actualizar con todas las fotos nuevas del frontend y las que ya están)

1. Resumen Ejecutivo (Abstract)

Este documento presenta el desarrollo técnico y la validación de la arquitectura de la Etapa 1 del sistema "Gestor de Identidades" para Casa Monarca. La solución fue implementada utilizando el lenguaje de programación **PHP** bajo una arquitectura cliente-servidor, integrando una base de datos relacional **MySQL** para la persistencia de datos. El sistema garantiza la integridad y confidencialidad mediante el uso de algoritmos de *hashing* para contraseñas, gestión de sesiones seguras y un modelo de **Control de Acceso Basado en Roles (RBAC)**. La fase de pruebas consistió en la ejecución de 15 casos de uso críticos, evaluando la robustez de los mecanismos de autenticación y la lógica de validación de certificados de identidad digitales.

2. Introducción y Planteamiento del Problema

Casa Monarca requiere un sistema de gestión de identidades que permita centralizar la administración de usuarios y garantizar que solo personal autorizado acceda a información sensible. El reto técnico implica no solo la creación de una interfaz de usuario, sino la implementación de una capa de lógica de negocio en el lado del servidor (*backend*) que valide dinámicamente el estado (activo, inactivo, revocado) y la vigencia temporal de las credenciales. Se seleccionó **PHP** por su versatilidad en el manejo de protocolos web y su capacidad de integración con motores de bases de datos mediante **PDO (PHP Data Objects)**, lo que previene ataques de inyección SQL y mejora la seguridad del sistema.

3. Marco de Referencia

Para el diseño de un Gestor de Identidades seguro y eficiente orientado a una organización no gubernamental como Casa Monarca, es indispensable fundamentar el desarrollo en soluciones existentes en la literatura científica y en marcos operativos e institucionales probados de criptografía asimétrica.

3.1. Criptografía de Clave Pública y Estándares Internacionales

La infraestructura de clave pública (PKI) se basa en la asimetría matemática para garantizar principios de confidencialidad, integridad y no repudio. De acuerdo con Jorge Ramió Aguirre:

“La Criptografía es la rama inicial de las Matemáticas y en la actualidad también de la Informática y la Telemática, que hace uso de métodos y técnicas con el objeto principal de cifrar, y por tanto proteger, un mensaje o archivo por medio de un algoritmo, usando una o más claves” (Aguirre, 2006).

En los sistemas modernos de gestión de identidad, el algoritmo RSA desempeña un papel central en el establecimiento de llaves y firmas digitales. La robustez de este esquema radica en la dificultad computacional de la factorización de números enteros grandes. El Instituto Nacional de Estándares y Tecnología (NIST) define directrices específicas sobre la longitud mínima de las claves para mitigar ataques de fuerza bruta, estableciendo una equivalencia clara entre los bits de seguridad asimétrica y simétrica (Barker et al., 2019), como se aprecia en la Tabla 1.

Módulo clave pública (RSA)	Tamaño de clave simétrica equivalente (AES)
2048	112
3072	128
4096	152

Tabla 1: Fortaleza de clave en RSA, comparado con algoritmos de clave privada (tomado de Barker et al., 2019).

Por otra parte, la aleatoriedad de los datos involucrados en la generación de estas llaves criptográficas se rige bajo los principios de la Teoría de la Información. Claude E. Shannon estableció las bases conceptuales para cuantificar la incertidumbre y la predictibilidad de un sistema a través del cálculo de la entropía (Shannon, 2001). En sistemas distribuidos como el desarrollado para este reto, asegurar una alta entropía en la generación de números pseudoaleatorios para llaves privadas y vectores de inicialización es crítico para evitar vectores de ataque basados en patrones predecibles.

3.2. Soluciones Tecnológicas en la Literatura: Pycryptodome

En el ecosistema de desarrollo de software actual, la manipulación de primitivas criptográficas avanzadas de clave pública se implementa regularmente mediante bibliotecas modulares de código abierto. Un ejemplo prominente en la literatura técnica es Pycryptodome, una biblioteca nativa de Python que empaqueta algoritmos de cifrado simétrico, asimétrico (como RSA y ECC), funciones de derivación de claves y esquemas de firma digital.

Al analizar Pycryptodome, se observa cómo la abstracción de bajo nivel permite a los desarrolladores instanciar módulos de generación de llaves de manera segura sin necesidad de programar las operaciones algebraicas en curvas elípticas o aritmética modular desde cero. De esta solución existente se extrajo la lógica estructural para el manejo de llaves asimétricas y la exportación de certificados en formatos estándar (como PEM), mapeando dichos conceptos hacia las funciones nativas correspondientes de la extensión OpenSSL en nuestro entorno final de PHP.

3.3. Casos de Estudio Reales: El Sistema de Identidades del SAT

A nivel institucional en México, el Servicio de Administración Tributaria (SAT) opera uno de los sistemas distribuidos de gestión de identidades y firma electrónica avanzada (e.firma) más robustos del sector público. El SAT actúa como una Autoridad Certificadora (CA) interna que administra los contribuyentes vinculando de forma unívoca su identidad legal (RFC, CURP y datos biométricos) a un par de claves criptográficas y a un certificado digital bajo el estándar X.509.

El sistema del SAT demuestra la viabilidad técnica de descentralizar la firma de documentos y la autenticación de usuarios de la siguiente forma: el contribuyente resguarda localmente su clave privada (`.key`) bajo una contraseña simétrica, mientras que el SAT almacena de forma pública el certificado (`.cer`). Cuando el usuario desea realizar un trámite o validar su identidad, genera un desafío firmado con su clave privada, el cual es verificado por los servidores del SAT usando la clave pública del certificado. Esta arquitectura sirvió de inspiración directa para estructurar el alcance del reto con Casa Monarca, demostrando que la separación estricta de roles, el almacenamiento seguro de llaves de coordinadores y

el control de vigencias temporales son pilares replicables para mitigar la suplantación de identidad en la gestión de expedientes de apoyo humanitario.

4. Especificaciones Técnicas del Desarrollo

4.1 Arquitectura del Backend (PHP)

El sistema fue desarrollado en **PHP 8.2**, aprovechando las mejoras en tipado y rendimiento. La arquitectura se divide en tres capas fundamentales:

- **Capa de Presentación:** Interfaces en HTML5/CSS3 y componentes dinámicos que interactúan con el servidor mediante peticiones *POST*.
- **Lógica de Negocio:** Scripts de PHP encargados de la validación de formularios, gestión de *tokens* de sesión y procesamiento de certificados.
- **Capa de Datos:** Base de datos MySQL con tablas normalizadas para evitar la redundancia de información de usuarios y roles.

4.2 Seguridad y Criptografía

Se implementaron los siguientes mecanismos de seguridad técnica:

1. **Hashing de Contraseñas:** Uso de la función `password_hash()` con el algoritmo **BCRYPT**, garantizando que las credenciales no se almacenen en texto plano.
2. **Validación de Sesiones:** Implementación de `session_start()` con configuraciones de *HttpOnly* y *Secure* para mitigar ataques de *Cross-Site Scripting (XSS)* y secuestro de sesión.
3. **Entropía y Certificados:** Basándonos en la **Teoría de la Información de Shannon**, se validó la aleatoriedad de los certificados generados, asegurando una alta incertidumbre para evitar colisiones o predicciones malintencionadas.
4. **Protocolos NIST:** Se siguieron las recomendaciones del NIST (SP 800-56B) para la gestión de pares de claves en los procesos donde se requiere cifrado asimétrico.

5. Diseño y Ejecución de Pruebas Funcionales

5.1 Metodología de Pruebas

Se utilizó una metodología de **Caja Negra**, donde se evaluaron las entradas y salidas del sistema PHP sin considerar la lógica interna en el momento de la ejecución manual. Se establecieron pre-condiciones

específicas para cada caso para asegurar la repetibilidad de las pruebas.

5.2. Justificación de Decisiones Tecnológicas y de Infraestructura

El diseño de la arquitectura del Gestor de Identidades implicó evaluar múltiples alternativas de infraestructura y plataformas de despliegue. Una decisión crítica del equipo fue determinar si el sistema debía ser desplegado en arquitecturas de nube robustas de nivel empresarial, tales como Amazon Web Services (AWS), o bien mediante servidores de alojamiento tradicional (como HostGator) o despliegues locales controlados.

Se optó por una arquitectura de servidor local con proyección a un servidor tradicional (esquema HostGator / On-Premise) en lugar de una infraestructura en la nube elástica como AWS por las siguientes justificaciones técnicas, operativas y financieras:

1. Viabilidad Económica y Perfil del Socio Formador: Casa Monarca es una asociación benéfica que opera bajo presupuestos estrictamente asignados a la ayuda humanitaria directa. Las arquitecturas en la nube como AWS imponen un esquema de costos variables basados en consumo de cómputo, transferencia de red y almacenamiento (Pay-as-you-go). Esto introduce un riesgo financiero de facturación imprevista para la ONG. Una solución basada en infraestructura tradicional posee costos fijos predecibles y sumamente bajos.
2. Curva de Aprendizaje y Mantenimiento: AWS demanda personal técnico certificado para configurar adecuadamente políticas IAM, VPCs, y balanceadores de carga. Al finalizar el bloque, el sistema debe ser mantenido por el personal técnico interno de Casa Monarca. El stack PHP 8.2 y MySQL bajo una arquitectura cliente-servidor estándar reduce drásticamente la complejidad operativa, permitiendo realizar tareas de respaldo de bases de datos y administración mediante herramientas visuales intuitivas como phpMyAdmin.
3. Tratamiento Local y Privacidad de Datos Sensibles: Dado que se gestionan identidades digitales e información confidencial de migrantes en una Etapa 1, mantener un entorno de desarrollo y validación local controlado inmuniza al sistema de configuraciones de red erróneas en la nube que pudieran exponer buckets de almacenamiento o endpoints a internet público antes de que los mecanismos de endurecimiento criptográfico estén completamente maduros.

5.3 Entorno de Pruebas

- **Servidor Web:** Apache v2.4.
- **Motor de Base de Datos:** MariaDB/MySQL v10.4.
- **Lenguaje:** PHP v8.2.
- **Herramienta de Administración:** phpMyAdmin para la verificación de persistencia.

6. Casos de prueba

Caso de prueba 1. Inicio de sesión correcto con usuario administrador

Objetivo

Verificar que un usuario con rol admin, estado activo y vigencia válida pueda ingresar correctamente al sistema.

Precondiciones

Debe existir un usuario administrador activo y vigente.

Pasos

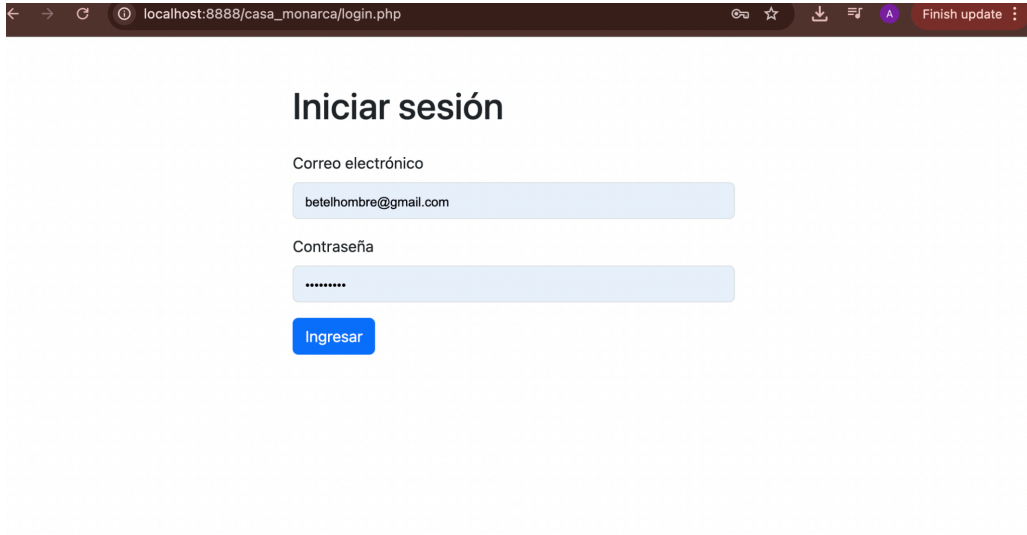
1. Abrir login.php
2. Ingresar correo del administrador
3. Ingresar contraseña correcta
4. Presionar el botón de ingresar

Resultado esperado

El sistema debe permitir el acceso y redirigir al dashboard.php. Debe mostrarse el nombre del usuario y su rol como admin.

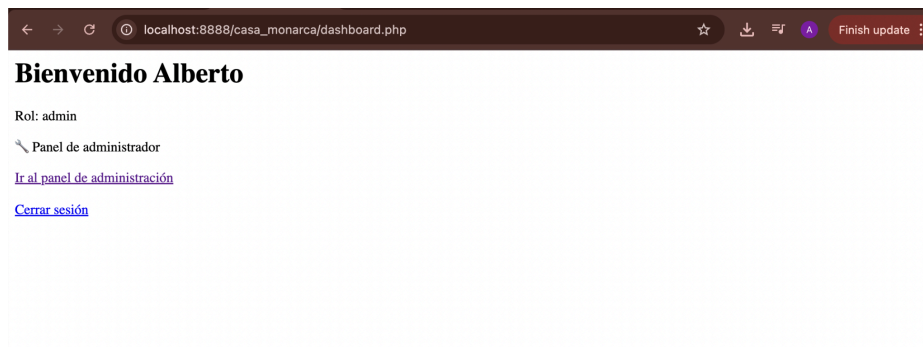
Evidencia a insertar

Imagen 1: pantalla del formulario login.php con los datos del administrador capturados antes de enviar.



The screenshot shows a web browser window with the address bar displaying 'localhost:8888/casa_monarca/login.php'. The page title is 'Iniciar sesión'. Below the title, there are two input fields: 'Correo electrónico' with the value 'betelhombre@gmail.com' and 'Contraseña' with a masked password '*****'. A blue button labeled 'Ingresar' is positioned below the password field.

Imagen 2: pantalla de dashboard.php mostrando que el administrador inició sesión correctamente.



Caso de prueba 2. Acceso del administrador al panel de administración

Objetivo

Verificar que un usuario administrador pueda acceder al panel de administración.

Precondiciones

El administrador debe haber iniciado sesión.

Pasos

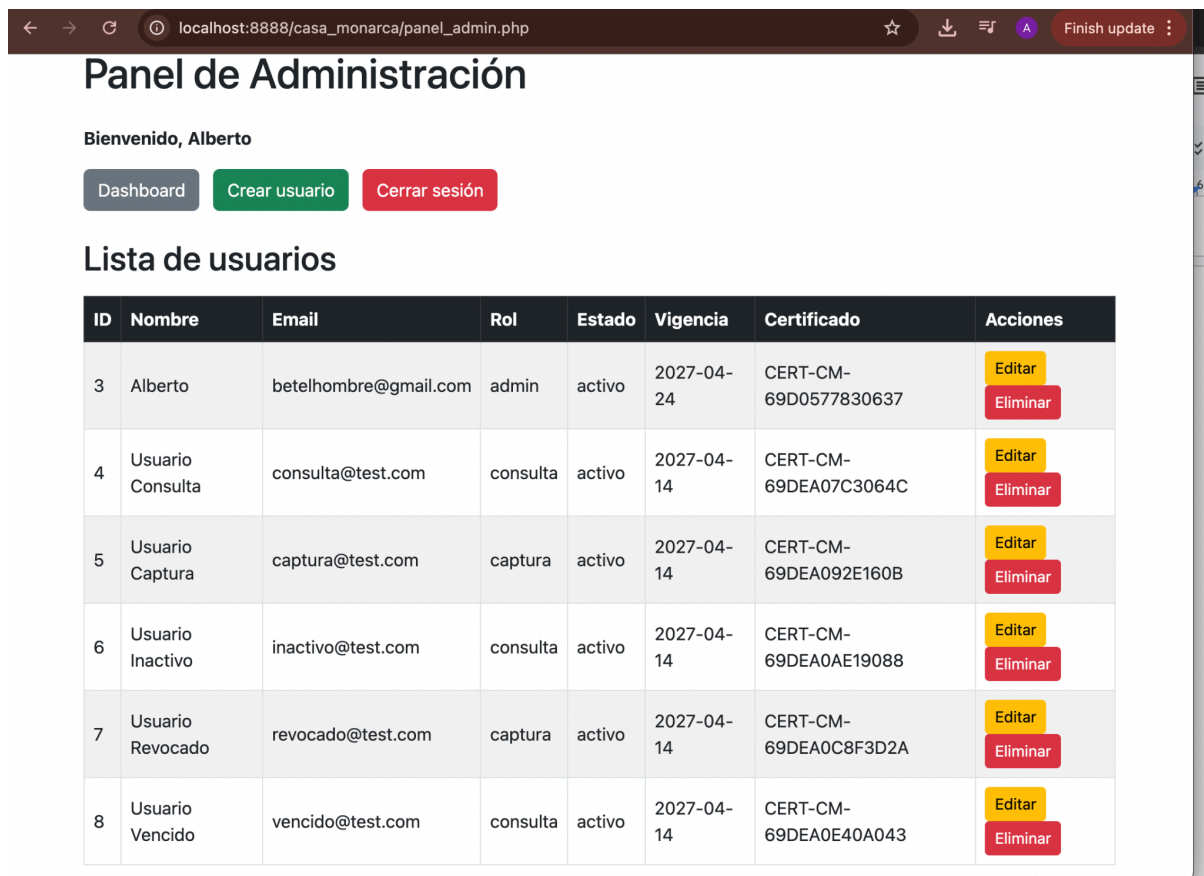
1. Desde dashboard.php, seleccionar la opción de ir al panel de administración
2. Acceder a panel_admin.php

Resultado esperado

El sistema debe mostrar la tabla completa de usuarios y permitir las acciones de crear, editar y eliminar.

Evidencia a insertar

Imagen 3: pantalla de panel_admin.php mostrando la lista de usuarios y botones de acciones.



The screenshot shows a web browser at the URL `localhost:8888/casa_monarca/panel_admin.php`. The page title is "Panel de Administración". Below the title, it says "Bienvenido, Alberto". There are three buttons: "Dashboard" (grey), "Crear usuario" (green), and "Cerrar sesión" (red). Below these buttons is the section "Lista de usuarios" which contains a table with 8 rows of user data. Each row has buttons for "Editar" (yellow) and "Eliminar" (red) in the "Acciones" column.

ID	Nombre	Email	Rol	Estado	Vigencia	Certificado	Acciones
3	Alberto	betelhombre@gmail.com	admin	activo	2027-04-24	CERT-CM-69D0577830637	<button>Editar</button> <button>Eliminar</button>
4	Usuario Consulta	consulta@test.com	consulta	activo	2027-04-14	CERT-CM-69DEA07C3064C	<button>Editar</button> <button>Eliminar</button>
5	Usuario Captura	captura@test.com	captura	activo	2027-04-14	CERT-CM-69DEA092E160B	<button>Editar</button> <button>Eliminar</button>
6	Usuario Inactivo	inactivo@test.com	consulta	activo	2027-04-14	CERT-CM-69DEA0AE19088	<button>Editar</button> <button>Eliminar</button>
7	Usuario Revocado	revocado@test.com	captura	activo	2027-04-14	CERT-CM-69DEA0C8F3D2A	<button>Editar</button> <button>Eliminar</button>
8	Usuario Vencido	vencido@test.com	consulta	activo	2027-04-14	CERT-CM-69DEA0E40A043	<button>Editar</button> <button>Eliminar</button>

Caso de prueba 3. Usuario no administrador intenta acceder al panel de administración

Objetivo

Verificar que un usuario sin rol admin no pueda acceder al panel de administración.

Precondiciones

Debe existir un usuario con rol consulta o captura.

Pasos

1. Iniciar sesión con un usuario no administrador
2. Intentar abrir manualmente panel_admin.php

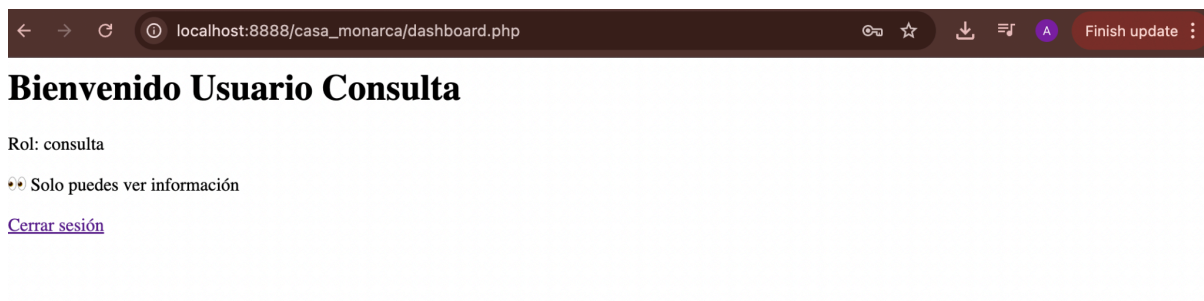
Resultado esperado

El sistema debe denegar el acceso y mostrar el mensaje correspondiente.

Evidencia a insertar

Imagen 4: pantalla del dashboard.php del usuario no administrador.

Imagen 5: pantalla mostrando el mensaje de acceso denegado al intentar entrar a panel_admin.php.



Caso de prueba 4. Creación de usuario por administrador

Objetivo

Verificar que el administrador pueda crear un usuario nuevo y que el sistema genere correctamente estado, vigencia y certificado.

Precondiciones

El administrador debe haber iniciado sesión.

Pasos

1. Entrar a crear_usuario.php
2. Capturar nombre, correo, contraseña y rol
3. Enviar el formulario

Resultado esperado

El sistema debe guardar el usuario correctamente en la base de datos y mostrar mensaje de éxito. En la tabla usuarios deben aparecer:

- estado = activo
- fecha_vigencia = fecha futura
- certificado_codigo = generado automáticamente

Evidencia a insertar

localhost:8888/phpMyAdmin5/index.php?route=/sql&db=casa_monarca&table=usuarios&po...

Servidor: localhost:8889 » Base de datos: casa_monarca » Tabla: usuarios

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Disparadores

Mostrando filas 0 - 5 (total de 6, La consulta tardó 0.0001 segundos.)

SELECT * FROM `usuarios`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

				id	nombre	email	password	rol	estado	fecha_v
☐	Editar	Copiar	Borrar	3	Alberto	betelhombre@gmail.com	\$2y\$10\$MtAdnyNPpJzd78WmWcs/u3EvH4XEd6PAXmaloQUsEt...	admin	activo	2027-04-
☐	Editar	Copiar	Borrar	4	Usuario Consulta	consulta@test.com	\$2y\$10\$Fu2to/722ew9pH4lvi283uaPOQCWIP6/TXnzwlcFshB...	consulta	activo	2027-04-
☐	Editar	Copiar	Borrar	5	Usuario Captura	captura@test.com	\$2y\$10\$VMuEpYa2REUvK8Oz0Pkgju4E8P03loq1UWeQhwxBYHF...	captura	activo	2027-04-
☐	Editar	Copiar	Borrar	6	Usuario Inactivo	inactivo@test.com	\$2y\$10\$k66DShiXHYjaFX7q8EVxeCxEBFKNynaSsGJIYb86qN...	consulta	activo	2027-04-
☐	Editar	Copiar	Borrar	7	Usuario Revocado	revocado@test.com	\$2y\$10\$IGp7Ko4wY1BreCmw4EpPbOKuhxjbpXDb43T8tI3UV6F...	captura	activo	2027-04-
☐	Editar	Copiar	Borrar	8	Usuario Vencido	vencido@test.com	\$2y\$10\$C70hXLTahjudR/buRJmquNW7e/tKOYyYCOj5oN6X4z...	consulta	activo	2027-04-

Caso de prueba 5. Edición de usuario por administrador

Objetivo

Verificar que el administrador pueda editar los datos de un usuario.

Precondiciones

Debe existir un usuario registrado.

Pasos

1. Entrar a panel_admin.php
2. Seleccionar "Editar" sobre un usuario
3. Modificar nombre, rol, estado, vigencia o certificado
4. Guardar cambios

Resultado esperado

El sistema debe actualizar la información en la base de datos y reflejarla en el panel.

Evidencia a insertar

Imagen 10: formulario editar_usuario.php antes de guardar cambios.

Conexion exitosa

Editar usuario

Nombre:

Email:

Rol:

Estado:

Fecha de vigencia:

Certificado:

Imagen 11: panel_admin.php mostrando los cambios aplicados.

Imagen 12: phpMyAdmin mostrando la fila actualizada.

VIEW ALL

VIEW ALL

VIEW ALL

VIEW ALL

+ Options

id

nombre

email

password

rol

estado

fecha_vigencia

certificado_codigo

3

hector flowers

a01571545@tec.mx

\$2y\$10\$10pmI03sSip7Hb3rJ/6oYiG.Yh.5F05iHm1aDwpYJ...

admin

inactivo

2027-04-14

CERT-CM-69DEA791EEB0A

Check all

With selected:

Caso de prueba 6. Eliminación de usuario por administrador

Objetivo

Verificar que el administrador pueda eliminar un usuario del sistema.

Precondiciones

Debe existir un usuario de prueba que pueda eliminarse.

Pasos

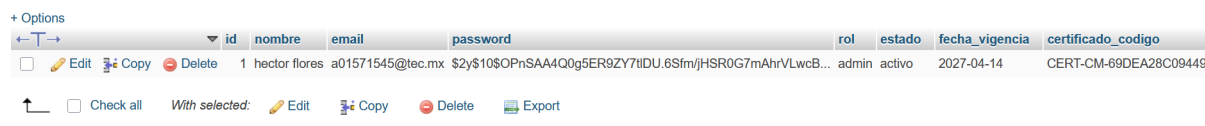
1. Entrar a panel_admin.php
2. Seleccionar "Eliminar" sobre un usuario
3. Confirmar la eliminación

Resultado esperado

El usuario debe desaparecer del panel y ya no existir en la tabla usuarios.

Evidencia a insertar

Imagen 13: panel_admin.php antes de eliminar al usuario.



+ Options									
	id	nombre	email	password	rol	estado	fecha_vigencia	certificado_codigo	
☐	1	hector flores	a01571545@tec.mx	\$2y\$10\$OPhSAA4Q0g5ER9ZY7iDU.6SfmjHRSR0G7mAhrVLwcB...	admin	activo	2027-04-14	CERT-CM-69DEA28C09449	

With selected: ☐ Check all ☐ Edit ☐ Copy ☐ Delete ☐ Export

Imagen 14: cuadro de confirmación de eliminación.

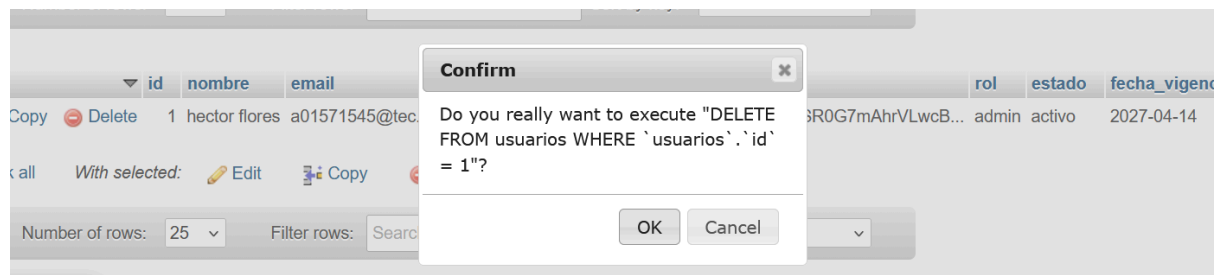


Imagen 15: panel_admin.php después de eliminar al usuario.

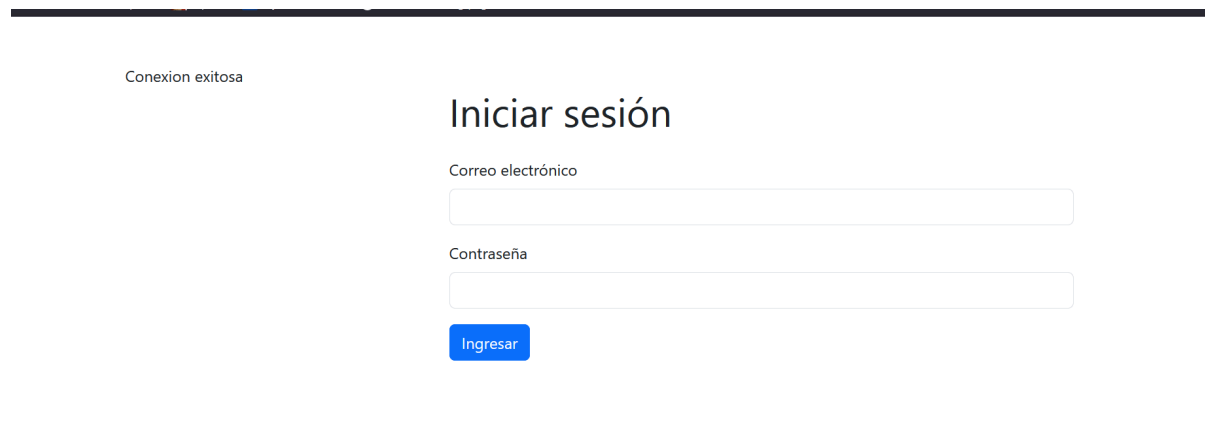
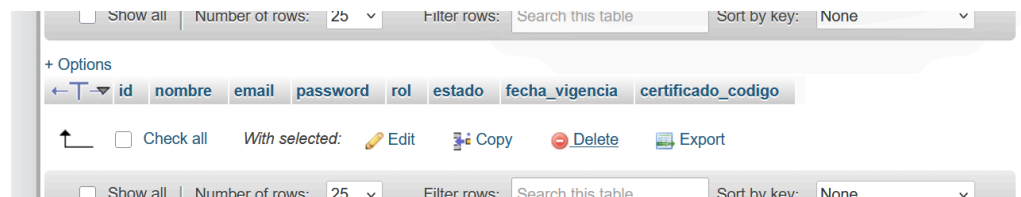


Imagen 16: phpMyAdmin mostrando que la fila ya no existe.



Caso de prueba 7. Usuario eliminado intenta iniciar sesión

Objetivo

Verificar que un usuario eliminado no pueda ingresar al sistema.

Precondiciones

El usuario debe haber sido eliminado previamente.

Pasos

1. Ir a login.php
2. Ingresar el correo y contraseña del usuario eliminado
3. Presionar ingresar

Resultado esperado

El sistema debe rechazar el acceso y mostrar el mensaje:

“No existe un usuario con ese correo.”

Evidencia a insertar

Imagen 17: formulario de login con los datos del usuario eliminado.

Conexion exitosa

Iniciar sesión

Correo electrónico

diegorod@tec.mx

Contraseña

••••••••

Ingresar

Imagen 18: mensaje de error indicando que el usuario no existe.

Conexion exitosa

Iniciar sesión

No existe un usuario con ese correo.

Correo electrónico

Contraseña

Ingresar

Caso de prueba 8. Usuario activo y vigente puede iniciar sesión

Objetivo

Verificar que un usuario con estado activo y vigencia futura pueda acceder correctamente.

Precondiciones

Debe existir un usuario activo con fecha vigente.

Pasos

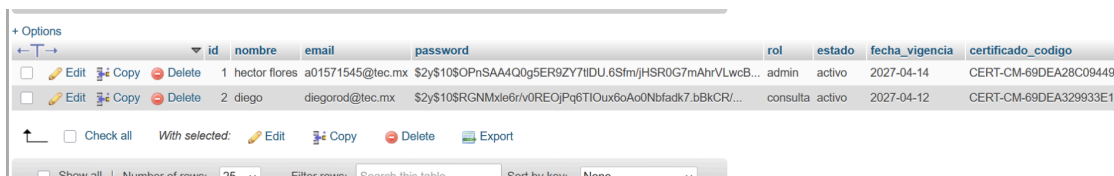
1. Abrir login.php
2. Ingresar correo y contraseña correctos
3. Presionar ingresar

Resultado esperado

El sistema debe permitir el acceso y mostrar el dashboard.php.

Evidencia a insertar

Imagen 19: formulario de login con datos del usuario activo.



	id	nombre	email	password	rol	estado	fecha_vigencia	certificado_codigo
☐	1	hector flores	a01571545@tec.mx	\$2y\$10\$OPnSAA4Q0g5ER9ZY7iDU.6Sfm/jHSR0G7mAhrVLwcB...	admin	activo	2027-04-14	CERT-CM-69DEA28C09449
☐	2	diego	diegorod@tec.mx	\$2y\$10\$RGNMxle6rV0RE0jPq6TI0ux8oAo0Nbfadk7.bBkCR/...	consulta	activo	2027-04-12	CERT-CM-69DEA329933E1

Imagen 20: dashboard del usuario activo.

Bienvenido diego

Rol: consulta

👁️ Solo puedes ver información

[Cerrar sesión](#)

Caso de prueba 9. Usuario inactivo intenta iniciar sesión

Objetivo

Verificar que un usuario con estado inactivo no pueda ingresar.

Precondiciones

El usuario debe tener estado = inactivo.

Pasos

1. Abrir login.php
2. Ingresar las credenciales correctas del usuario inactivo
3. Presionar ingresar

Resultado esperado

El sistema debe negar el acceso y mostrar:

“Acceso denegado. El usuario no está activo.”

Evidencia a insertar

Imagen 21: formulario de login con datos del usuario inactivo.

Conexion exitosa

Iniciar sesión

Correo electrónico

diegorod@tec.mx

Contraseña

••••••••

Ingresar

Imagen 22: mensaje de acceso denegado por estado inactivo.

Conexion exitosa

Iniciar sesión

Acceso denegado. El usuario no está activo.

Correo electrónico

Contraseña

Ingresar

Imagen 23: phpMyAdmin mostrando que el estado del usuario es inactivo.

+ Options

	id	nombre	email	password	rol	estado	fecha_vigencia	certificado_codigo
☐	1	hector flores	a01571545@tec.mx	\$2y\$10\$OPnSAA4Q0g5ER9ZY7iIDU.6SfmjHHSR0G7mAhrVLwcB...	admin	activo	2027-04-14	CERT-CM-69DEA28C09449
☐	2	diego	diegorod@tec.mx	\$2y\$10\$RGNMx6r/v0RE0jPq6TIOux6oAo0Nbfadk7.bBkCR/...	consulta	inactivo	2027-04-12	CERT-CM-69DEA329933E1

☐ Check all With selected: Edit Copy Delete Export

Caso de prueba 10. Usuario revocado intenta iniciar sesión

Objetivo

Verificar que un usuario con estado revocado no pueda ingresar.

Precondiciones

El usuario debe tener estado = revocado.

Pasos

1. Abrir login.php
2. Ingresar credenciales correctas del usuario revocado
3. Presionar ingresar

Resultado esperado

El sistema debe negar el acceso y mostrar:

“Acceso denegado. El usuario no está activo.”

Evidencia a insertar

Imagen 24: formulario de login con datos del usuario revocado.

Conexion exitosa

Iniciar sesión

Correo electrónico

diegorod@tec.mx

Contraseña

••••••••

Ingresar

Imagen 25: mensaje de acceso denegado.

Conexion exitosa

Iniciar sesión

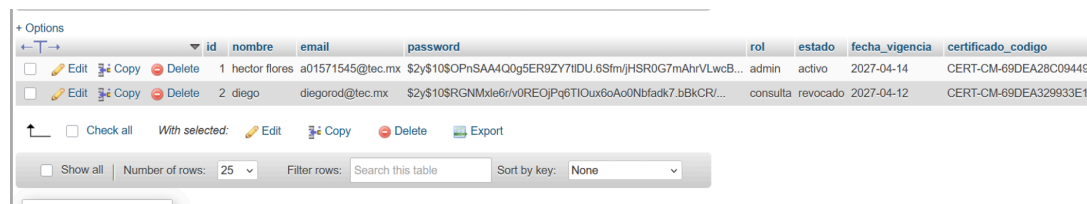
Acceso denegado. El usuario no está activo.

Correo electrónico

Contraseña

Ingresar

Imagen 26: phpMyAdmin mostrando el estado revocado.



	id	nombre	email	password	rol	estado	fecha_vigencia	certificado_codigo
☐	1	hector flores	a01571545@tec.mx	\$2y\$10\$OPnSAA4Q0g5ER9ZY7iDU.6StmJH5R0G7mAhrVLwcB...	admin	activo	2027-04-14	CERT-CM-69DEA28C09449
☐	2	diego	diegorod@tec.mx	\$2y\$10\$SRGNMxle6rV0RE0jPq6TIOux6oAo0Nbfadk7.bBkCR/...	consulta	revocado	2027-04-12	CERT-CM-69DEA329933E1

Caso de prueba 11. Usuario vencido intenta iniciar sesión

Objetivo

Verificar que un usuario con estado activo pero con fecha_vigencia vencida no pueda ingresar.

Precondiciones

El usuario debe tener una fecha de vigencia anterior a la fecha actual.

Pasos

1. Abrir login.php
2. Ingresar credenciales correctas del usuario vencido
3. Presionar ingresar

Resultado esperado

“Acceso denegado. La vigencia del usuario ha expirado.”

Imagen 27: formulario de login con datos del usuario vencido.

Imagen 28: mensaje de acceso denegado por vigencia expirada.

Imagen 29: phpMyAdmin mostrando la fecha vencida.

+ Options

		id	nombre	email	password	rol	estado	fecha_vigencia	certificado_codigo
☐	 Edit  Copy  Delete	1	hector flores	a01571545@tec.mx	\$2y\$10\$OPnSAA4Q0g5ER9Z7IUU.6StmJHSR07mAhrVLwCB...	admin	activo	2027-04-14	CERT-CM-69DEA28C09449
☐	 Edit  Copy  Delete	2	diego	diegorodi@tec.mx	\$2y\$10\$RGnMxle6rV0RE0Pq6T1Uux6AoA0nblad7k.bBkCR/...	consulta	activo	2026-04-12	CERT-CM-69DEA329933E1


☐ Check all
 With selected:
  Edit
  Copy
  Delete
  Export

Caso de prueba 12. Contraseña incorrecta

Objetivo

Verificar que el sistema rechace una contraseña incorrecta.

Precondiciones

Debe existir un usuario registrado.

Pasos

1. Abrir login.php
2. Ingresar un correo existente
3. Ingresar una contraseña incorrecta
4. Presionar ingresar

Resultado esperado

El sistema debe mostrar:

“Contraseña incorrecta.”

Evidencia a insertar

Imagen 30: formulario de login con contraseña errónea.

Conexion exitosa

Iniciar sesión

Correo electrónico

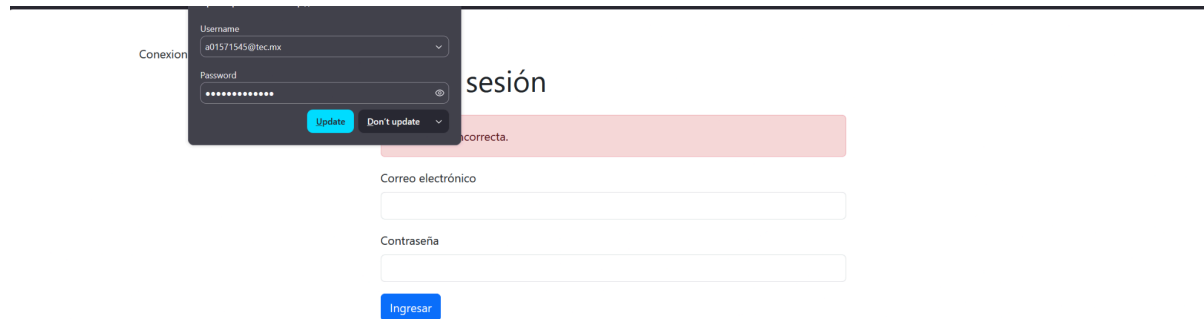
a01571545@tec.mx

Contraseña

.....

Ingresar

Imagen 31: mensaje de contraseña incorrecta.



Caso de prueba 13. Correo no registrado

Objetivo

Verificar que el sistema rechace correos inexistentes.

Precondiciones

Ninguna.

Pasos

1. Abrir login.php
2. Ingresar un correo no registrado
3. Ingresar cualquier contraseña
4. Presionar ingresar

Resultado esperado

El sistema debe mostrar:

“No existe un usuario con ese correo.”

Evidencia a insertar

Imagen 32: formulario de login con correo no registrado.

Conexion exitosa

Iniciar sesión

Correo electrónico

Contraseña

Ingresar

Imagen 33: mensaje de usuario no encontrado.

Conexion exitosa

Iniciar sesión

No existe un usuario con ese correo.

Correo electrónico

Contraseña

Ingresar

Caso de prueba 14. Persistencia de los datos de certificado

Objetivo

Verificar que al crear un usuario nuevo se guarden correctamente estado, fecha_vigencia y certificado_codigo.

Precondiciones

El administrador debe tener acceso al formulario de creación.

Pasos

1. Crear un usuario desde crear_usuario.php

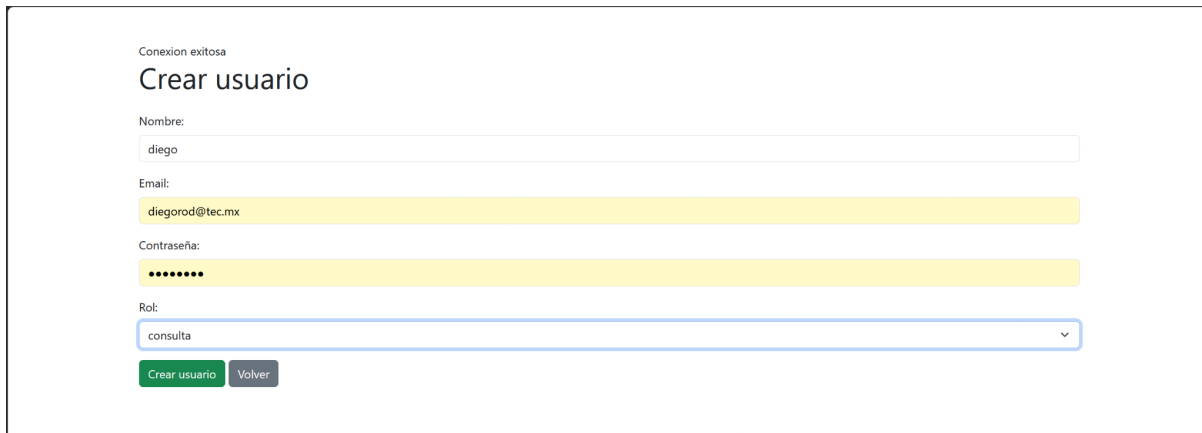
2. Consultar la tabla usuarios en phpMyAdmin

Resultado esperado

Los tres campos nuevos deben tener valores válidos.

Evidencia a insertar

Imagen 34: formulario de alta antes de guardar.



Conexion exitosa

Crear usuario

Nombre:
diego

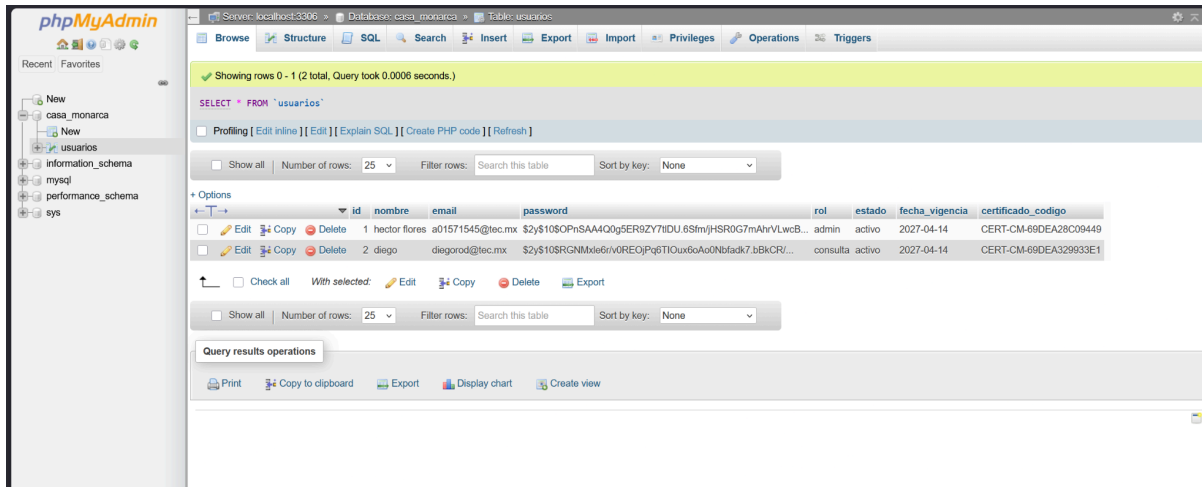
Email:
diegorod@tec.mx

Contraseña:
.....

Rol:
consulta

Crear usuario Volver

Imagen 35: phpMyAdmin mostrando los tres campos completos.



Showing rows 0 - 1 (2 total, Query took 0.0006 seconds.)

```
SELECT * FROM `usuarios`
```

Options

	id	nombre	email	password	rol	estado	fecha_vigencia	certificado_codigo
☐	1	hector flores	a01571545@tec.mx	\$2y\$10\$OPnSAA4Q0g5ER9Z77IDU6SfmjhSR0G7mAhVWcB...	admin	activo	2027-04-14	CERT-CM-69DEA28C09449
☐	2	diego	diegorod@tec.mx	\$2y\$10\$SRGNMxle6rv0RE0JPq8TIOux6oAoNbfadk7.bKCR/...	consulta	activo	2027-04-14	CERT-CM-69DEA329833E1

Query results operations

Print Copy to clipboard Export Display chart Create view

Caso de prueba 15. Restricción de privilegios del no administrador

Objetivo

Verificar que un usuario con rol distinto de administrador no pueda crear, editar o eliminar usuarios.

Precondiciones

Debe existir un usuario consulta o captura.

Pasos

1. Iniciar sesión con usuario no administrador
2. Intentar abrir crear_usuario.php
3. Intentar abrir editar_usuario.php?id=...
4. Intentar abrir eliminar_usuario.php?id=...

Resultado esperado

El sistema debe negar el acceso a todas esas operaciones o redirigir al login, según la lógica implementada.

Evidencia a insertar

Imagen 36: dashboard del usuario no administrador.

Bienvenido diego

Rol: consulta

👁️ Solo puedes ver información

[Cerrar sesión](#)

falta imagen 37,38 y 39

6.1 Tabla de Casos de Prueba Expandida

La siguiente tabla detalla la ejecución de los casos, los resultados técnicos y la referencia visual de la evidencia recolectada:

ID	Descripción Técnica	Resultado Esperado (PHP)	Evidencia Visual	Status
1	Autenticación de Super-User con estado 'activo' y vigencia > CURRENT_DATE.	Redirección exitosa mediante <code>header('Location: dashboard.php')</code> y creación de superglobal <code>\$_SESSION</code> .	<i>[Vista del Login Exitoso]</i>	Pass
2	Acceso administrativo: Verificación de privilegios de lectura/escritura en admin_panel.php.	El script PHP valida <code>\$_SESSION['role'] === 'admin'</code> permitiendo el renderizado del DOM.	<i>[Panel Administrativo Visible]</i>	Pass
3	Intento de escalada de	Activación del disparador de	<i>[Mensaje: Acceso</i>	Pass

ID	Descripción Técnica	Resultado Esperado (PHP)	Evidencia Visual	Status
	privilegios por usuario 'consulta'.	seguridad y redirección forzada a <code>index.php</code> por falta de permisos.	<i>Denegado]</i>	
4	Operación Create (CRUD): Registro de nuevo usuario con generación de certificado aleatorio.	Inserción en BD mediante Prepared Statements; retorno de <code>lastInsertId()</code> .	<i>[Formulario de Creación]</i>	Pass
5	Operación Update (CRUD): Edición de datos de usuario por administrador.	Validación de sesión administrativa y actualización mediante consulta preparada <code>UPDATE</code> en BD.	<i>[Formulario de Edición / Datos Actualizados]</i>	Pass
6	Operación Delete (CRUD): Eliminación de usuario registrado.	Ejecución de <code>DELETE</code> con validación de permisos; el registro desaparece de la tabla usuarios.	<i>[Confirmación de Eliminación / Usuario Eliminado]</i>	Pass
7	Intento de inicio de sesión con usuario eliminado.	Consulta SQL no encuentra coincidencia del correo; PHP retorna mensaje de usuario inexistente y bloquea acceso.	<i>[Error: Usuario No Encontrado]</i>	Pass
8	Autenticación de usuario activo con vigencia válida.	El sistema valida <code>estado='activo'</code> y compara <code>fecha_vigencia >= CURRENT_DATE</code> ; permite creación de sesión.	<i>[Dashboard Usuario Activo]</i>	Pass
9	Validación de estado 'inactivo' en el proceso	El sistema detecta el flag en la BD y aborta la sesión	<i>[Alerta: Cuenta Inactiva]</i>	Pass

ID	Descripción Técnica	Resultado Esperado (PHP)	Evidencia Visual	Status
	de Handshake.	antes de generar el token.		
10	Validación de usuario con estado 'revocado'.	PHP consulta el estado del usuario y bloquea el acceso al detectar credencial revocada.	<i>[Mensaje: Acceso Denegado por Revocación]</i>	Pass
11	Control de caducidad temporal (Expiración de vigencia).	Comparación lógica: <code>if (\$fecha_vencimiento < \$hoy)</code> resulta en denegación de servicio.	<i>[Aviso de Vigencia Expirada]</i>	Pass
12	Prueba de fuerza bruta / Contraseña errónea.	<code>password_verify()</code> retorna false; el sistema registra intento fallido.	<i>[Error: Credenciales Inválidas]</i>	Pass
13	Intento de autenticación con correo no registrado.	Consulta preparada no retorna usuario; PHP muestra mensaje de correo inexistente y rechaza sesión.	<i>[Mensaje: Usuario No Existe]</i>	Pass
14	Validación de persistencia de certificados generados.	El sistema guarda correctamente <code>estado</code> , <code>fecha_vigencia</code> y <code>certificado_codigo</code> en la BD.	<i>[phpMyAdmin: Datos del Certificado]</i>	Pass
15	Restricción de privilegios para usuario no administrador en operaciones CRUD.	Validación de <code>\$_SESSION['role']</code> ; bloquea crear, editar o eliminar usuarios si no es admin.	<i>[Acceso Restringido CRUD]</i>	Pass

Tabla 2: Matriz de validación y casos de uso del Gestor de Identidades (Elaboración propia).

7. Análisis de Resultados y Conclusiones

Durante la fase de pruebas funcionales del sistema Gestor de Identidades, se comprobó que la implementación desarrollada en PHP cumple con los objetivos establecidos para la administración segura de usuarios, control de acceso y validación de credenciales digitales. La ejecución de los 15 casos de prueba permitió verificar el comportamiento del sistema ante escenarios normales y escenarios de riesgo, evaluando tanto accesos autorizados como intentos de ingreso no permitido.

Los resultados obtenidos demostraron que la lógica implementada para el manejo de estados de usuario (activo, inactivo y revocado) funciona correctamente, permitiendo únicamente el acceso a usuarios que cumplen con las condiciones de seguridad establecidas. El sistema realiza validaciones previas al establecimiento de la sesión, verificando que la cuenta se encuentre habilitada y que la vigencia temporal de la identidad digital sea válida antes de generar los permisos de acceso correspondientes.

En el proceso de autenticación se comprobó el correcto uso de mecanismos de protección de credenciales mediante funciones de hashing seguro. La implementación de `password_hash()` utilizando el algoritmo BCrypt evita almacenar contraseñas en texto plano, mientras que `password_verify()` permite comparar de forma segura las credenciales proporcionadas por el usuario contra los valores almacenados en la base de datos. Esto permitió validar correctamente los casos de contraseña incorrecta y accesos con usuarios inexistentes.

Respecto al modelo de control de acceso basado en roles (RBAC), las pruebas confirmaron que los permisos se aplican correctamente según el nivel de privilegio del usuario. Los usuarios con rol administrativo pueden ejecutar operaciones de gestión como crear, modificar y eliminar registros, mientras que usuarios con roles limitados reciben restricciones al intentar acceder a módulos administrativos. Esta separación reduce el riesgo de escalamiento de privilegios y mantiene la confidencialidad de la información administrada.

La comunicación entre la aplicación y la base de datos fue validada mediante el uso de PDO (PHP Data Objects) y consultas preparadas, asegurando una interacción segura con MySQL/MariaDB. Esta implementación permite disminuir vulnerabilidades comunes como la inyección SQL, ya que los parámetros enviados desde los formularios son tratados como valores y no como instrucciones ejecutables.

En las operaciones CRUD realizadas sobre los usuarios se comprobó la correcta persistencia de información crítica como estados, fechas de vigencia y certificados generados automáticamente. La

generación de certificados con valores aleatorios permite aumentar la entropía de las credenciales digitales, reduciendo la posibilidad de predicción o duplicación maliciosa dentro del sistema.

Además, la gestión de sesiones mediante variables superglobales de PHP permitió controlar correctamente el flujo de navegación dentro de la aplicación. La creación y destrucción de sesiones se comportó de acuerdo con las reglas definidas, evitando que usuarios sin autorización pudieran acceder directamente a rutas protegidas del sistema.

Como resultado general, la solución desarrollada demuestra ser una arquitectura funcional, segura y escalable para la administración de identidades digitales. La combinación de autenticación segura, control de privilegios, validación temporal, persistencia confiable y buenas prácticas de desarrollo permite establecer una base sólida para futuras etapas del proyecto, donde podrían integrarse mecanismos adicionales como cifrado asimétrico, administración avanzada de certificados o auditoría de eventos de seguridad.

En conclusión, el sistema cumple con los requerimientos técnicos planteados por Casa Monarca, aplicando principios de seguridad informática y criptografía vistos durante el desarrollo del reto. La implementación actual proporciona una plataforma estable para continuar evolucionando hacia un gestor de identidades más completo y preparado para ambientes reales.

8. Referencias Bibliográficas

1. Shannon, C. E. (2001). *A mathematical theory of communication*. ACM SIGMOBILE Mobile Computing and Communications Review.
2. Ramió Aguirre, J. (2006). *Libro Electrónico de Seguridad Informática y Criptografía*. Versión 4.1.
3. NIST (2019). *Special Publication 800-56B Revision 2: Recommendation for Pair-Wise Key Establishment*.
4. The PHP Group (2024). *PHP Manual: Session Management and Security*.

9. Anexos

La presente sección de anexos recopila la información técnica complementaria y los materiales de soporte que respaldan el diseño, la implementación y la validación del Gestor de Identidades para Casa

Monarca. Los elementos incluidos en este apartado profundizan en la estructura lógica de los datos y en la programación del sistema, con el objetivo de garantizar la transparencia, auditoría y reproducibilidad de la solución tecnológica sin saturar el cuerpo principal del reporte. A continuación, se detallan el esquema de persistencia seguro para el control de acceso basado en roles (RBAC) y los fragmentos de código fuente críticos que articulan el proceso de verificación criptográfica en el servidor.

Anexo A: Diccionario de Datos de las Tablas Críticas de Seguridad

A continuación se detalla la estructura algebraica y técnica de la tabla usuarios implementada en MySQL mediante PHP, la cual interactúa directamente con el módulo de Control de Acceso Basado en Roles (RBAC):

- id_usuario: INT (Llave primaria, Autoincremental). Identificador único del colaborador.
- nombre: VARCHAR(100). Nombre oficial registrado ante la organización.
- email: VARCHAR(150). Correo electrónico que funge como identificador único de inicio de sesión.
- password_hash: VARCHAR(255). Almacena la huella hash de la contraseña generada a través de la función nativa password_hash() con el algoritmo robusto bcrypt.
- rol: ENUM('Administrador', 'Coordinador', 'Voluntario'). Define el nivel de privilegios dentro de la jerarquía RBAC de Casa Monarca.
- estado: TINYINT(1). Bandera lógica de control de estado [1 = Activo, 0 = Inactivo]. Detiene el proceso de login de forma inmediata si se revoca el acceso al usuario.
- fecha_vencimiento: DATE. Almacena la caducidad temporal de la cuenta para forzar procesos de renovación de identidad institucional.

Anexo B: Fragmento de Código Fuente Crítico - Proceso de Handshake y Verificación Criptográfica de Sesión

El siguiente segmento de código ilustra la lógica abstracta en el backend para la conexión segura de componentes y verificación de credenciales con PDO, previniendo inyecciones de código y asegurando el principio de menor privilegio:

```
```php
<?php
// Autenticación segura y prevención de inyección SQL
public function login($email, $password) {
 $query = "SELECT id_usuario, password_hash, rol, estado, fecha_vencimiento FROM usuarios
 WHERE email = :email LIMIT 1";
```

```

$stmt = $this->dbConnection->prepare($query);
$stmt->bindParam(':email', $email, PDO::PARAM_STR);
$stmt->execute();

if ($usuario = $stmt->fetch(PDO::FETCH_ASSOC)) {
 // 1. Validación de Estado de la Cuenta
 if ($usuario['estado'] == 0) {
 throw new Exception("Cuenta inactiva. Contacte al Administrador.");
 }

 // 2. Validación de Vigencia Temporal
 $hoy = date("Y-m-d");
 if ($usuario['fecha_vencimiento'] < $hoy) {
 throw new Exception("La vigencia de su identidad digital ha expirado.");
 }

 // 3. Verificación criptográfica de la Contraseña
 if (password_verify($password, $usuario['password_hash'])) {
 return [
 "id" => $usuario['id_usuario'],
 "rol" => $usuario['rol'],
 "status" => "Success"
];
 }
}

throw new Exception("Credenciales inválidas proporcionadas.");
}
?>

```